

‘Desperately’ Seeking Research on Agile Requirements in the Context of Large-Scale Agile Projects

Knut H. Rolland
Westerdals Oslo School of Arts,
Communication and Technology &
SINTEF ICT
Schweigaardsgt. 14
0185 Oslo, Norway

+47 92882331
rolknu@westerdals.no

ABSTRACT

In this paper, I propose an agenda for undertaking research on requirements engineering in the context of large-scale agile software development projects. In so doing, I draw from my own experience as a practitioner in large-scale agile software development projects, as well as a review of relevant literature. Based on this, the paper reflects on two central questions: what are the main challenges related to requirements in large-scale agile software development? And, what are the relevant themes for researching requirements in large-scale agile projects? The paper concludes by proposing some overall research questions for future research.

Categories and Subject Descriptors

D.2.1 [Requirements/Specifications]: Elicitation methods

General Terms

Management

Keywords

Requirements Engineering, Agile Requirements, Agile Software Development, Large-scale projects, Information Systems Development, Empirical research

1. INTRODUCTION

Requirements engineering (RE) has long been an important theme of research in the fields of software engineering and information systems. In relation to agile software development, the very notion of ‘requirements engineering’ is often avoided because of its obvious connotations towards relying heavily on documentation rather than continuous interaction with users. Recently, however, the topic has attracted some attention in both research [1] and in more practitioner-oriented texts [2].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Request permissions from Permissions@acm.org.
XP 2015 Workshops, May 25 - 29, 2015, Helsinki, Finland
© 2015 ACM. ISBN 978-1-4503-3409-9/15/05 \$15.00
DOI: <http://dx.doi.org/10.1145/2764979.2764984>

While often based on more informal descriptions and less sequential than traditional RE processes [3], agile requirements are also often documented in terms of user stories, through the use tools like Atlassian JIRA, and various forms of sketches. Furthermore, like more traditional approaches, agile software development projects also struggles with interdependencies between requirements, establishing architectures, and experiencing challenging communication with stakeholders [1, 4].

In particular, large-scale agile projects [5], need to sort out requirements with potential dependencies across different teams creating a need for new forms of coordination mechanisms that in small-scale agile and the typical agile practices do not exist. Hence, there is a need for looking deeper into the agile requirements processes and practices – and in particular for large-scale agile projects.

This short paper aims at contributing by proposing an agenda for researching agile requirements in large-scale projects, and asks: *What are the main challenges related to agile requirements?* And, *what are important themes for research on agile requirements in large-scale projects?*

In trying to answer these questions, the paper first reviews relevant empirical studies of requirements processes and agile requirements. Then, in section 3, the paper offers some experiences from two large-scale agile projects in which the author was involved as a practitioner. Based on insights in current literature and my own experiences, the paper then offers three themes for research that relates to the main challenges identified.

2. RELEVANT LITERATURE

2.1 Empirical studies of requirements practices and RE-processes

The challenge of RE has long been well known, and has been studied from various perspectives in both the fields of software engineering and information systems. While the body of literature in general is large, there exist fewer empirically grounded studies [4, 6, 7, 8, 9, 10]. Based on this empirical literature in broad strokes it is emphasized that requirements are *emergent*, *socio-technical*, and tend to be fraught with a variety of *communication and coordination* challenges. Although these are rather overall phenomena, I will argue that these both reflect finding from current empirical studies and also can serve as a starting point for discussing agile requirements.

2.2 The emergent nature of requirements and processes

The emergent nature of requirements implies that it seems notoriously difficult to establish a stable and complete set of requirements early on in the process. New requirements tend to emerge out of changing users needs, unknown technical details, and different stakeholders influencing the project. Over time, as developers and other projects member learn more about the application domain and the customer's organization, it is also likely that requirements will become more nuanced and detailed.

This emergent nature of requirements and the RE process is illustrated in Curtis and colleagues field study of large-scale software development [7]. From their study of 17 large software projects they found that "fluctuating" or "conflicting" requirements were main challenges in all projects. Rather than a consequence of poorly planned projects, the emergent nature of requirements stems from a wide range of different factors. The authors find that the emergent nature of requirements are due to external aspects such as changes of customers' needs, changes in technologies, and regulatory schemes. Requirements also change in relation to a complex context internal to the business organizations. These include organizational politics, procurement processes, as well as conflicting requirements among different user groups and stakeholders.

Holmström and Sawyer [9] also finds that requirements are emergent, and illuminates how developers and projects tend to 'black box' or simply avoid addressing issues that are perceived to fluctuate all together. Rather than simply 'gathered' their case study shows that requirements are negotiated. This is often because of conflicting interest or views of different stakeholders. Moreover, their case study also reveals that developers tend to perceive the client organization as a homogeneous entity in terms of having common views and user needs. This, of course, simplifies the RE process, however, at the same time this practice risks not uncovering crucial nuances and changes, and hence not undertaking important modifications along the process.

Karlsson et al. [4] draws from a qualitative study based on eight cases, and identifies many different problems (and practices) related to requirements and the RE process. Interestingly, in their study they find that while some embrace emergent requirements as a "fact of software development" others see this as a threat to stability of a new product. In one of the case companies the informants reported that a common practice was to not finish off more than 60-90% of a particular requirement in order to open for customer feedback before spending all resources. Henceforth both the Karlsson et al. [4] and the Holström and Sawyer [9] study show that development projects' practices are heavily influenced by the emergent nature of requirement – yet in very different ways. Likewise, Ovaska et al. [11], also illustrate the emergent character in that requirements were discovered continuously and managed through "heroic achievements" meaning working according to the situation" (p. 20).

Interestingly, similar findings are – as mentioned above, found in the aforementioned Curtis et al.'s study from the late 1980s and studies conducted over 20 years later. This indicates that the emergent nature of requirements is not so much related to techniques, tools or software technology in itself as organizational and social issues. However, it could also indicate that the basic approaches (and their failures) for requirements processes in projects have not changed much over the years.

2.3 The intrinsic socio-technical character of requirements

A different characteristic of requirements and RE-processes in large-scale projects is the almost intrinsic socio-technical nature of such processes. While requirements are traditionally defined as variations of "what a system should do" [3: p. 4] reflecting the technical features and properties of a future software artifact, the process of eliciting, analyzing and validating requirements can be argued to be largely a social and political process [6, 11]. In this way, although the end result in terms of a functional or non-functional requirement represents a technical artifact, the process of producing these representations typically involves multiple stakeholders, negotiations and conflicts.

For example, in their study Holmström and Sawyer [9: p. 9] states that "[All] respondents reported that it was helpful for conflicts to arise during development projects and that this was much better than having conflicts surface after the projects had ended. Conflict while not always pleasant, was seen as the best means to provide all actors the opportunity to learn and consider multiple sides of an issue...". Importantly this indicates that if projects try to ignore the social and organizational aspects of requirements process, learning and stabilization of requirements might be undermined. Along these lines, the authors argue that focus on opposing arguments and views can be fruitful for requirements processes. Implicit in the authors theorizing of requirements processes as social construction on technology (SCOT), describing how different actors – or stakeholders are shaping technology, the larger the project the more likely it is that a lot of effort is needed for negotiating and stabilizing requirements.

Likewise, Bjarnason et al. [6] argues that a "weak vision of overall goal" is a main cause of communication breakdowns in relation to requirements. In light of Holmström and Sawyer's [9] arguments above, lack of a process through which negotiations and stakeholders can discuss their opposing views, a weak vision can indeed be a consequence.

Arguably, however, there is perhaps also a tendency in existing literature to not take the technological artifacts seriously enough. Especially, when the case in most organizations is that typically new IS and digital services need to connect and integrate with existing systems and infrastructures [13, 14]. In relation to requirements processes in large-scale projects this is important since in could imply that a large amount of requirements are related to integration, interface standards, formats, and 'middleware' services like SOA. This has consequences for how the requirements process is organized. As many companies tend to leave the 'business parts' of the organization to specify requirements, they are likely not to have the competence for specifying the more technically-oriented requirements [8]. Thus, leaving the decisions to developers with less competence on user needs, and increasing the 'gap of communication' [4].

2.4 Coordination and communication challenges

Based on the previous section, the theme of "coordination and communication" seems to be highly relevant. A main finding in current studies is that there are typically severe challenges involved in coordination and communication of requirements. All ready in the 1980s the Curtis et al. [7: p. 1282] study emphasized that "[Organizational] boundaries hindered understanding the requirements" and that "the complexity of the customer interface hindered the establishment of stable requirements and increased the communication and negotiation cost of the project". While

traditional RE-processes has a strong focus on documentation and representation of requirements, Curtis et al. [7] find that the amount of communication in large-scale projects are not reduced by documentation.

In more recent studies some of these findings are confirmed and investigated further. Karlsson et al. [4] identifies a “communication gap between marketing staff and developers”. A particular problem seems to be when one part of an organization (e.g. marketing) are writing the requirements specification independently and then hands it over to a different department to implement those requirements (e.g. IT development). According to the study some organizations applied “mediators” that reformulated the requirements from marketing into something that developers would want. Furthermore, the study finds that different stakeholders have different views on what constituted a “good requirement” [4: p. 594].

In general, communicating with users is often perceived as one of the more difficult tasks that also requires the analyst to understand the more tacit aspects of work practices and meaning – and not only what users articulate: “It is often the case that users find it difficult to articulate their requirements. To this end, a requirements engineer can resort to eliciting information about the tasks users currently perform...” [15: p.39]. Henceforth, in large-scale projects when the number of different user communities increases, this becomes a formidable task.

A related issue discussed in Bjarnason et al. [10] finds a communication gaps between different stakeholders in software development to be root causes of what they identify as the problem of overscoping. Here overscoping is defined as setting a scope that is too large for the current resources available.

2.5 Requirements in an agile context

There are currently few empirical studies of RE-processes especially geared towards agile software development. However, a notable exception is Ramesh and colleagues’ qualitative study of practices in 16 different software organizations in the USA [1]. In their paper [1: p. 452], the authors investigated the following three aspects of agile requirements:

1. What RE practices are adopted in software development in agile environments?
2. What problems are encountered when these practices are used?
3. Do these practices mitigate or increase requirements risks?

We discuss the two first ones here. Concerning the first research question, Ramesh et al. [1] concludes that agile RE-practice does not follow the traditional linear and sequential RE-process or the assumptions regarding rigor and accuracy. However, agile RE also involves all aspects of requirements elicitation, analysis and verification [cf. 3] as found in traditional RE. More accurately, find six different RE-practices in agile projects: face-to-face communication, iterative RE, constant planning, extreme prioritization, prototyping, and reviews and tests. Interestingly, many of these practices are at least in theory more in line with the dynamics of requirements processes discussed in the previous section. For example, the practices of ‘iterative RE’ and ‘constant planning’ seem to be appropriate in light of the emergent nature of requirements. As noted by Ramesh et al. [1] the more a project learn about the artifact being developed, the more user stories are typically added. Thus, taking into account the emergent nature of requirements. Similarly, the agile practice of ‘constant planning’

in light of new feedback for stakeholders make it possible to learn and adjust requirements during the development process.

Regarding the socio-technical character of requirements processes, agile requirements practices as reported by Ramesh et al. [1], many of the agile requirement practices offers possibilities for a larger degree of negotiation with stakeholders. For example, ‘prototyping’ facilitates negotiating with users the detailed requirements for the artifact. More over, ‘face-to-face’ communication would often be necessary when negotiating details and for sorting out conflicts as reported by Holmström and Sawyer [9]. This is in most cases harder and a more tedious process when simply relying on documentation. Considering the challenge of communication and coordination, it seems likely that use of ‘review meetings and test’, ‘face-to-face communication’, and ‘prototyping’ to be an advantage. However, also in large-scale agile projects when having many development teams this seems to be a tough challenge [16].

The Ramesh et al. [1] study does also identify new challenges introduced by these agile requirements practices. These are ‘cost and schedule estimation’, ‘inadequate architecture’, ‘neglected non-functionals’, ‘minimum documentation’, ‘customer access and participation’, ‘prioritization on single dimension’, and ‘no requirements verification’. In relation to the more generic challenges from the literature discussion above, there are some potential linkages worth examining further. Firstly, it is not only single requirements that are emerging during the process, but also software architecture. Thus, it seems probable that agile requirements practices tend to down play interdependencies between requirements (i.e. user stories) so that resulting architectures become inadequate. Regarding the socio-technical character of requirements processes, an interesting aspect is the perceived lack of ‘customer access and participation’ and the problem of ‘prioritization on single dimension’. These tendencies can severely endanger the possibilities for negotiating requirement throughout the process with various stakeholders. As Ramesh et al. [1] notes, also customer consensus is especially important especially when we know that in large-scale projects this is unlikely to be the case. Reflecting on the challenge of communication and coordination, the challenges identified by the study also show that this is still an issue with agile requirements. In particular, ‘no requirements verification’ and again the lack of ‘customer access and participation’ contribute to this.

To summarize, based on existing literature, it seems that agile requirements practices partly meets the more generic challenges involved with requirements processes and that they simultaneously also still suffer from the very same dynamics.

3. SOME STORIES FROM THE TRENCHES

Working as a practitioner, I have over the last years been involved in two large-scale agile software development project. By large-scale, I refer to projects that involve more than one development team as defined by Dingsøyr and Moe [5]. Both projects had at times 13-14 teams working in parallel. Additionally, the projects involved a large number of participants (more than 200) from both the customer organization and suppliers. A second characteristic that arguably made the projects ‘large-scale’ was the sheer number of user stories to be implemented. One of the projects had close to 2500 user stories defined up-front before the suppliers were enrolled in the project. Moreover, a third aspect of ‘large-scale’ can be attributed to the resulting IS cutting across many different contexts and organizational boundaries [13].

In both cases I worked on the customer side of the project mainly with analyzing user needs, managing and specifying early interaction concepts, enterprise architecture and user stories. The projects had many different characteristics, technical challenges and specific contextual ramifications – however, both projects had similar challenges when it came to managing the agile requirements process.

Firstly, in both cases it proved *tremendously challenging to verify user stories largely written by different domain experts working in the customer's organization*. Because of the scale of the projects the resulting software also had many different user groups and stakeholders. This made it hard to verify whether or not the user stories actually related to the most pressing user needs. In one of the projects, we arranged various interviews and workshops with users and discovered that some of the user stories were more related to features in the existing IS that the users thought of as insufficient functionality or lack of integration – and hence not necessary the most appropriate requirements for a new IS. On the other hand, we also found that some of the more pressing user needs were not covered in the user stories. Many of the user stories were written a long time before they were attempted coded, and the authors were not always available for the project. Other user stories we had completely misunderstood, as we had not been able to communicate directly with some of the user groups. Arguably, this illustrates some of the challenges with agile requirements in large-scale projects as no individual, or possibly not even a team, from the customer is able to precisely and timely sort out the details of heterogeneous user needs. Such situations underscore the emergent nature of requirements in general, and show that the practice of ‘black-boxing’ can indeed also become a problem in agile requirements [9]. According to my own experiences, the challenges to agile requirements verification found by Ramesh et al [1] are also very evident. It also seems likely that these issues become even more pronounced in large-scale projects.

Secondly, in large-scale agile projects there is often *hard to get an overview of hundreds – sometimes thousands of user stories, and in particular, different kinds of interdependencies between them*. In both projects there I participated, a continuous and notoriously difficult aspect was to uncover functional and/or technical interdependencies between user stories. For example, in both projects an aspect of the new IS was to implement a new digital work process for receiving, storing, and handling ‘cases’. In so doing, we had to figure out how to best sequence the user stories for the purpose of producing something in each sprint that could be testable as well as being an artifact that end-users could relate to and understand during demos and other more ad-hoc interactions. Although state-of-the-art process modeling was conducted, these models became so complex that re-prioritizing the product backlog implied around a week of work in order to evaluate whether this would have undesirable consequences for testing and end-user interaction or not. And usually it had. Moreover, lack of any complete overview often led to re-prioritization of entire collections of user stories. Re-prioritizing of user stories on this scale had consequences for setting up the software architecture and integrations to other systems. More often than not, the software architects had to re-think the architecture since different collections of user stories required different aspects of the architecture to be in place. This situation potentially explains Ramesh et al. [1: p. 463]’s point that “[The] architecture chosen by the development team during the early cycles may become inappropriate or inadequate as newer requirements become known”. Re-prioritizing collections of user

stories with different technical and functional interdependencies make it hard to distribute user stories across several development teams – without risking the teams to become too interdependent creating coordination complexities [16]. This also underscores the ‘political ecology’ [12] of agile requirements; when some actors in the organization have long defined and prioritized collections of user stories, this situation can hence make it hard for others to completely dismiss their work. Henceforth, this point also aligns well with the findings in the Ramesh et al. [1] study of agile requirements practices. Furthermore, as many agile methods, techniques and experiences are crafted for and related to ‘small-scale’ agile projects this typically sets specific expectations among stakeholders. For example, as agile methods are interpreted as more flexible than waterfall approach – stakeholders expect possibilities for re-prioritizing and changing requirements as the project proceeds. Hence, this can set the stage for well-known ‘expectation failures’ [17]. To summarize, large-scale agile projects seem to have an almost intrinsic problem with organizing and prioritizing a product backlog.

Thirdly, large-scale agile projects tend to involve *increased complexity because of an installed base of existing IS and architectures*. In particular, one of the projects involved various kinds of interdependencies with an installed-base of over 300 different IS. Parts of this installed base was maintained by different suppliers and supported by different stakeholders in the organization, making it a time consuming process of mobilizing resources and stakeholders to actually prioritize changes in these existing systems as needed by the new project. This had several consequences for the agile project. First, this situation made it necessary to specify more detailed requirements in terms of standardized interfaces and user stories with acceptance criteria weeks ahead of the sprint planning and the sprint there they were implemented. Henceforth, the agility or flexibility to do changes in the actual sprint planning meeting was greatly reduced. Secondly, it became more challenging for the participants in the project to find the relevant persons in the customer’s organization for clarifications regarding user stories, user needs and different juridical ramifications. This is an aspect of the requirements process that is seldom covered in both agile requirements literature and RE literature in general. However, while the challenges concerned with many stakeholders have been illuminated in existing literature [4], there is a lack of studies that look at the consequences of existing installed base and architecture for the agile requirements process and also agile software development in general.

4. AGENDA FOR RESEARCH

Based on the discussion above the following overall themes for future research can be suggested:

- **Theme 1:** Organizing of agile requirements and architecture evolution.
- **Theme 2:** Adaptation of agile requirements practices in large-scale projects that cut across boundaries.
- **Theme 3:** Theorizing the role of agile requirements practices and processes in large-scale projects.

The first theme underscores the importance of practices and processes for organizing agile requirements in ways that both ensures development of new features – and that simultaneously ‘grows’ an appropriate architecture. In large-scale agile projects the issue of architecture is likely to become more apparent because of the sheer number of user stories to be implemented, multiple teams as well as a wide range of different integration issues with other systems. Arguably, this issue is both supported

by current practitioner-oriented literature [18] as well as empirical research [1]. Based on the challenges identified above then, it seems that organizing of agile requirements in large-scale projects are caught in a dilemma: on the one hand agile projects would want to delay a too detailed specification of requirements to preserve flexibility. On the other hand, however, they also want collections of specific requirements that are more specified in order to develop a stable architecture because of multiple teams, complex testing and interdependencies. Potential research questions in relation to this theme can be:

- How are interdependencies between requirements managed in large-scale agile projects and what are the consequences for team coordination, testing and user communication?
- How are agile requirements organized in large-scale agile projects and what are the consequences for architecture?
- What agile requirements practices would reduce the risk of developing inadequate architectures?

The second theme, *adaptation of agile requirements practices in large-scale projects that cut across boundaries* highlights another important aspect of Re-processes in large-scale agile projects. As evidence from the empirical literature suggests [4, 6, 7, 9], it is especially challenging with requirements processes that cut across different departments and user groups in organizations that more often than not have different interpretations of requirements, of what requirements should be prioritized, and even conflicting requirements and interpretations. Arguably, although the agile practices of describing requirements on a rather high-level of detail and user friendly way using user stories works fairly well in small-scale projects (with only one team), the above discussion shows that this practice can become radically more problematic in large-scale efforts. Henceforth, there is a need for research that empirically investigates how this is practiced in large-scale projects. Moreover, this situation also is likely to cause problems for inter-team coordination in large-scale projects as user stories developed by one team can have unknown dependencies to a wider architecture and other user stories developed by neighboring teams. Thus, while there exists empirical studies of dynamics within teams [19], little is known about inter-team communication and coordination. A different aspect even less researched is the influence of an installed base of existing interconnected IS that increasingly is the new context that software development in organizations faces [14] and how this new context influences the agile requirements process. Organizations are not developing stand-alone systems anymore and new digital services in public sector for instance, relies upon a wide range of existing platforms, systems and standards already in place. Requirements processes are therefore not only likely to involve more stakeholders and become more technically challenging, but a larger amount of requirements are likely to be related to these existing systems and standards – that in turn can both facilitate and hinder agile practices in projects. On a more general level then, there is also a need for empirically investigate and theorizing upon the special types of adaptation related to agile requirements in large-scale projects [20].

Relevant research questions here can be:

- What are the current practices related to agile requirements and how are they shaped in interaction with different stakeholders' needs and interests in large-scale projects?

- What are the mechanisms for coordinating and communicating requirements across teams in large-scale agile projects?
- How should agile projects organize RE-processes in order to account for diverging stakeholder needs and reduce risks in development teams?
- How does an existing installed-base of systems, platforms and standards facilitate and hinder agile requirements processes and practices?

Regarding the third theme, *'theorizing the role of agile requirements practices and processes in large-scale projects'* this is linked to previous calls for more rigorous studies [21]. My point here is twofold. The first is that studies of agile requirements in large-scale projects also need to be more theoretically informed. For example, Moe et al. [19] in their study of agile teamwork draw on Dickinson and McIntyre's teamwork model for explaining team performance in agile teams. Similarly, Holmström and Sawyer [9] draw on the SCOT (social construction of technology) framework in order to shed light on the RE-process, and in particular the emergent character of requirements and how requirements are shaped and negotiated over time with by stakeholders. In the same way, studies of agile requirements need to be theoretically informed. This increases the generalizability of empirical studies and lead to discussion on an arguably more important analytical level rather than more arbitrary empirical details. My second point is that empirical studies of agile requirements practices in large-scale projects need also to lead to new theorizing. This can be done in many different ways. For example, through grounded theory [4], through synthesizing existing literature and theories [22], or through importing theories and concepts from other fields like for example the concept of 'communities-of-practice' used in the fields of knowledge management and organizational learning [16].

5. CONCLUDING REMARKS

This paper has only shortly surfaced the landscape of RE in large-scale agile software development projects. This rather short paper does not however give a complete overview of the literature nor do I claim that the themes and proposed research questions are exhaustive. My aim here is simply to illuminate some themes and possible research questions for embarking on important empirical research on agile requirements in larger software projects.

Arguably, RE in the context of large-scale agile projects is an increasingly important issue as much existing studies are concerned with RE in context of waterfall approaches and RE in context of small-scale agile projects. Furthermore, RE techniques as for instance user stories is often geared towards small-scale agile software development, and does not necessarily scale in larger projects. Hence, more of the 'same' research and more of the 'same' techniques are not necessarily the way to go forward. IT seems important that we do not fall for the temptation to extrapolate techniques and tools that work in the small to a potentially radically different large-scale agile project. This point is equally important for academics as well as practitioners.

6. ACKNOWLEDGEMENT

This article was written in the project Agile 2.0, which is supported by the Research council of Norway through grant 236759/O30, and by the companies Kantega, Kongsberg Defence & Aerospace, Sopra Steria and Sticos.

7. REFERENCES

- [1] Ramesh, Balasubramaniam, Lan Cao, and Richard Baskerville. Agile requirements engineering practices and challenges: an empirical study. *Information Systems Journal* 20, no. 5 (2010): 449-480.
- [2] Leffingwell, D. 2010. *Agile software requirements: lean requirements practices for teams, programs, and the enterprise*. Addison-Wesley Professional.
- [3] Sommerville, I., & Sawyer, P. 1997. *Requirements engineering: a good practice guide*. John Wiley & Sons, Inc..
- [4] Karlsson, Lena, Åsa G. Dahlstedt, Björn Regnell, Johan Natt och Dag, and Anne Persson. Requirements engineering challenges in market-driven software development—An interview study with practitioners. *Information and Software technology* 49, no. 6 (2007): 588-604.
- [5] Dingsøyr, Torgeir, and Nils Brede Moe. Research challenges in large-scale agile software development. *ACM SIGSOFT Software Engineering Notes* 38, no. 5 (2013): 38-39.
- [6] Bjarnason, Elizabeth, Krzysztof Wnuk, and Björn Regnell. Requirements are slipping through the gaps—A case study on causes & effects of communication gaps in large-scale software development. In *Requirements Engineering Conference (RE), 2011 19th IEEE International*, pp. 37-46. IEEE, 2011.
- [7] Curtis, Bill, Herb Krasner, and Neil Iscoe. A field study of the software design process for large systems. *Communications of the ACM* 31, no. 11 (1988): 1268-1287.
- [8] El Emam, Khaled, and Nazim H. Madhavji. A field study of requirements engineering practices in information systems development. In *Requirements Engineering, 1995., Proceedings of the Second IEEE International Symposium on*, pp. 68-80. IEEE, 1995.
- [9] Holmström, J., & Sawyer, S. 2011. Requirements engineering blinders: exploring information systems developers' black-boxing of the emergent character of requirements. *European Journal of Information Systems*, 20(1), 34-47.
- [10] Bjarnason, E., Wnuk, K., & Regnell, B. Are you biting off more than you can chew? A case study on causes and effects of overscoping in large-scale software engineering. *Information and Software Technology*, 54, no. 10 (2012), 1107-1124.
- [11] Ovaska, Päivi, Matti Rossi, and Kari Smolander. Filtering, negotiating and shifting in the understanding of information system requirements. *Scandinavian Journal of Information Systems* 17, no. 1 (2005): 31-66.
- [12] Bergman, Mark, John Leslie King, and Kalle Lyytinen. Large-scale requirements analysis revisited: The need for understanding the political ecology of requirements engineering. *Requirements Engineering* 7, no. 3 (2002): 152-171.
- [13] Hepso, Vidar, Eric Monteiro, and Knut H. Rolland. Ecologies of e-Infrastructures. *Journal of the Association for Information Systems* 10, no. 5 (2009): 430-446.
- [14] Tilson, David, Kalle Lyytinen, and Carsten Sørensen. Research commentary-digital infrastructures: the missing IS research agenda. *Information Systems Research* 21, no. 4 (2010): 748-759.
- [15] Nuseibeh, Bashar, and Steve Easterbrook. Requirements engineering: a roadmap. In *Proceedings of the Conference on the Future of Software Engineering*, pp. 35-46. ACM, 2000.
- [16] Paasivaara, Maria, and Casper Lassenius. Communities of practice in a large distributed agile software development organization—Case Ericsson. *Information and Software Technology* 56, no. 12 (2014): 1556-1577.
- [17] Lyytinen, Kalle, and Rudy Hirschheim. Information systems failures—a survey and classification of the empirical literature. *Oxford Surveys of Information Technology*, vol. 4 (1987): 257-309.
- [18] Kniberg, Henrik. *Scrum and XP from the Trenches*. Lulu.com, 2007.
- [19] Moe, Nils Brede, Torgeir Dingsøyr, and Tore Dybå. A teamwork model for understanding an agile team: A case study of a Scrum project. *Information and Software Technology* 52, no. 5 (2010): 480-491.
- [20] Fitzgerald, Brian, Gerard Hartnett, and Kieran Conboy. Customising agile methods to software practices at Intel Shannon. *European Journal of Information Systems* 15, no. 2 (2006): 200-213.
- [21] Abrahamsson, Pekka, Kieran Conboy, and Xiaofeng Wang. "Lots done, more to do": the current state of agile systems development research. *European Journal of Information Systems*, 18 (2009): 281-284.
- [22] Lyytinen, Kalle, and Gregory M. Rose. Information system development agility as organizational learning. *European Journal of Information Systems* 15, no. 2 (2006): 183-199.